R7021E Advanced Robotics and AI

Lab 3: autonomous exploration with RRT*

Group 6

Dominic Addo (domadd-2) and Abdalrahman Nasser (abdnas-2)

1. The plan and the implementation

The first plan used the frontier detector to group frontier cells into clusters. It would choose a point from each cluster, run a separate goal-directed RRT* for the six nearest points, and score the paths. That version worked in the small Gazebo maze, but its navigation node reached 638 lines and 29 tunable values. Several mechanisms had been added to patch individual failures, and we could no longer explain all of them.

The implemented version uses next-best view (NBV). One RRT* tree grows from the robot through known free space. Its leaves are candidate viewing positions. This reuses the tree to produce reachable candidates and removes clustering, a separate reachability search and one RRT* run per candidate. The navigation node is 330 lines with 13 tunable values.

2. ROS 2 nodes, topics and workflow

slam_toolbox publishes the occupancy map and the map-to-odometry transform. The frontier detector turns the map into frontier cells. Our navigation_node subscribes to the map, frontiers and TF, grows and scores the tree, then publishes a nav_msgs/Path. The provided path_follower_node turns that path into /cmd_vel. The robot publishes /scan and /odom to slam_toolbox. Navigation debug topics show the tree, goal and inflated map in RViz.

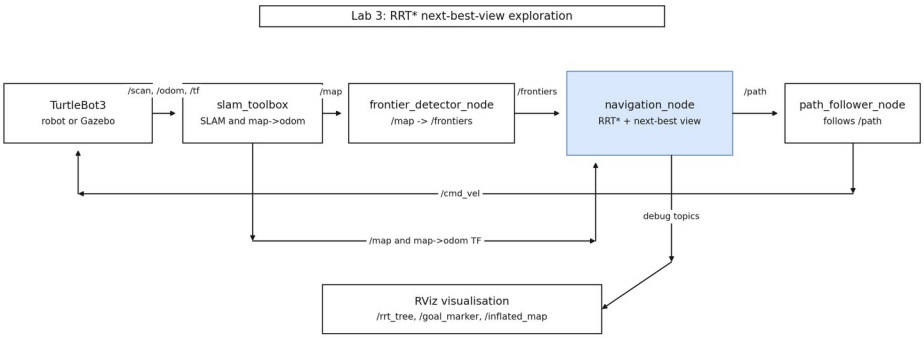


Figure 1: ROS 2 nodes and topics in the implemented exploration stack.

3. Planner and information gain

RRT* samples cells that are known free and outside the inflated occupied mask. Occupied cells are inflated by 0.105 m for the Burger footprint, rounded up to three 0.05 m cells. Each edge is checked at half-cell intervals. The tree grows from the robot

pose, or from the nearest free cell if the pose is inside the inflated padding; its nodes are connected to that root through known free space.

```
I(q) = number of frontier cells within 0.75 m of candidate q
```

I(q) counts frontier-cell centres within 0.75 m of q. It is a proximity heuristic, not a line-of-sight test, so frontier cells behind walls can still count.

```
H(q) = L(q) - w I(q) + c turn(q)
```

In words: the stored tree cost L raises the score, nearby frontier cells lower it, and a large first turn adds a cost. Rewiring changes a node parent but does not refresh cached costs for its descendants, so L can be stale and the selected score is not guaranteed to match the actual branch length. We used w = 0.10 m per frontier cell and c = 0.15 m/rad. A candidate needed at least 7 frontier cells during the Task 4 hall run; the selected Task 5 attempt used min_gain = 8.

```
tree <- RRT*(known free space, robot pose)
for each leaf q: count I(q); discard q if I(q) < min_gain
score remaining leaves with H(q); publish the lowest-score branch
drive the path; replan on arrival, stall or path age
```

The node plans on a timer, not inside the map callback. It uses map-to-base_link TF because SLAM can correct the pose relative to odometry.

4. Results in Gazebo and the hall

In the Gazebo small maze on 2026-10-11, the first NBV run reached 90 percent of its final mapped area in 168 s, with no stalls or back-offs. In the larger maze it repeatedly chose unreachable frontier leaves. Remembering a stalled goal improved the next run, but it still plateaued for about 8.5 minutes.

The hall runs were on 2026-10-13. Task 4 used min_gain = 7 and an inflation radius of 0.105 m. Task 5 attempt 3 used min_gain = 8 and 0.12 m. Both radii round to three 0.05 m cells, so the cleaner attempt 3 result cannot be attributed to a larger effective padding.

The Task 1 hall baseline mapped 17.8 m^2 in 216 s, with 90 percent of its final area reached at 151 s.

Hall run	Mapped area []	90% [s]	Stopped growing [s]	Plans	Stalls
Task 4, min_gain 7	19.8	148	316	31	7
Task 5, attempt 3, min_gain 8	20.6	125	205	15	0

Attempt 3 made the cleanest map and reached 90 percent fastest. Attempt 2 mapped 21.6 m^2, but its walls were rotated and doubled by SLAM drift. Some of its extra area was map smear, not additional maze.

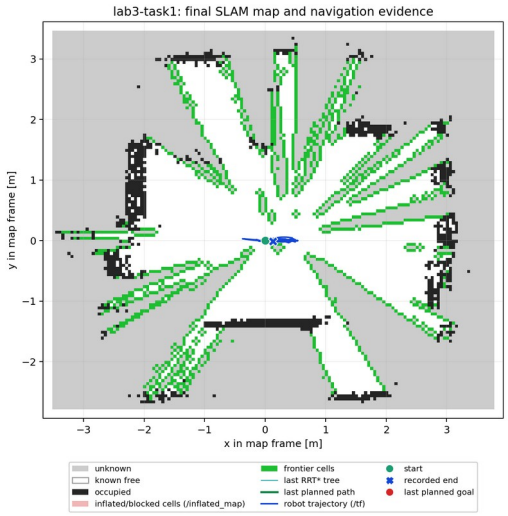


Figure 2: Task 1 hall baseline map.



Figure 3: Task 5 attempt 3. The walls are straight and the map outline is closed.

5. Information gain comparison

After the hall run, we compared three information weights in the 5.6 m Gazebo maze, with two runs per value. A nonzero weight improved the score over choosing the nearest leaf. The 0.10 and 0.30 run ranges overlap, so we cannot claim that one is faster.

info_weight [m/cell]	Mean score [s]	Mean plans	Mean stalls	Mean back-offs
0.00	933	185	1.5	4
0.10	764	49	0	0
0.30	783	41	1	5

All six runs mapped 29.2m^2 to 30.9 m^2 of the 31.4 m^2 maze. The 0.10 setting had no stalls or back-offs in either run and is as good as 0.30 within this comparison.

6. Problems during real-time operation

During the hall run, the robot clock was 12.6 s to 17.5 s behind the laptop. slam_toolbox logged full queues and scans earlier than the TF cache in Task 4, while scan delivery stayed at 4.6 to 4.9 Hz. Clock offset was the likely cause. In Gazebo, the follower froze at corners and could creep without reaching its stop condition. A laser stop, a 0.10 m back-off after 2 s blocked, slower turns and disabled loop closure addressed the observed failures. Task 5 attempt 3 stopped growing at 205 s with no stalls or back-offs. Its visualization video is included as a separate MP4 file. The single-tree NBV planner was easier to explain and worked best in the small maze. The larger maze exposed unreachable frontiers and long plateaus.